

Moon Units — Installation Guide

Prerequisites

Before installing Moon Units, ensure you have the following:

Requirement	Details
Docker	Installed and running
AWS CLI	Configured with credentials for a GoDaddy non-PCI AWS account
GitHub PAT	Personal access token with repo scope
gh CLI	GitHub CLI installed and authenticated (gh auth login)
Jira access	Atlassian API token + your Jira email
GoCode credentials	Your existing GoCode token (starts with sk-)
Artifactory credentials	Optional — only needed if your project pulls JFrog dependencies

Warning: Do not generate a new GoCode token if you already have one in use. Each user can only have two active tokens at a time — generating a new one may revoke a token you're using elsewhere.

Step 1: Download the **mu** Binary

Download the correct binary for your platform from the latest nightly release.

macOS (Apple Silicon)

```
# Download the binary
curl -LO https://github.com/gdcorp-
infosec/moonunit/releases/latest/download/mu-darwin-arm64
```

Linux (arm64)

```
curl -LO https://github.com/gdcorp-
infosec/moonunit/releases/latest/download/mu-linux-arm64
```

Linux (x86_64)

```
curl -LO https://github.com/gdcorp-  
infosec/moonunit/releases/latest/download/mu-linux-x64
```

Windows (x64)

Download `mu-windows-x64.exe` from the latest nightly release.

Step 2: Make the Binary Executable

macOS / Linux

```
chmod +x mu-darwin-arm64
```

(Replace `mu-darwin-arm64` with your platform's filename.)

Step 3: Remove macOS Quarantine (macOS Only)

macOS blocks unsigned binaries downloaded from the internet. Clear the quarantine attribute:

```
xattr -d com.apple.quarantine mu-darwin-arm64
```

Note: If you skip this step, macOS will show "'mu' could not be verified" and refuse to run the binary.

Step 4: Move to Your PATH

Move the binary somewhere on your `$PATH` and rename it to `mu`:

```
mv mu-darwin-arm64 ~/.local/bin/mu
```

Make sure `~/.local/bin` is in your PATH. If not, add it:

```
# Add to ~/.zshrc or ~/.bashrc  
export PATH="$HOME/.local/bin:$PATH"
```

Then reload your shell:

```
source ~/.zshrc
```

Step 5: Verify Installation

```
mu version
```

You should see a build SHA and timestamp confirming the installation.

Step 6: Configure Credentials

Run the interactive setup wizard:

```
mu init
```

The wizard will prompt you for:

Credential	Where to get it
GoCode token	Your existing token starting with <code>sk-</code>
GitHub token	Run <code>gh auth token</code> to retrieve it, or use your PAT
Jira email	Your Atlassian account email
Jira API token	Generate at https://id.atlassian.com/manage-profile/security/api-tokens
Artifactory credentials	Optional — press Enter to skip, or let it auto-detect

Credentials are saved to `~/.config/mu/mu.env`.

GoCode Credentials — How to Obtain

GoCode is GoDaddy's internal gateway (powered by GoCaaS) for accessing multiple LLMs (Claude, GPT, etc.) from your IDE, CLI, or notebook. Your GoCode API token authorizes `mu` to call AI models on your behalf.

Prerequisite: GoCode requires a **GoDaddy VPN connection**.

If You Already Have a GoCode Token

Use your existing token. You can find it in:

- Your shell environment: `echo $GOCODE_API_TOKEN`
- Your IDE plugin settings (e.g., VS Code GoCode extension)
- The GoCode web portal where you originally generated it

Important: Do NOT generate a new token if you already have one active. Each user can only have **two active tokens** at a time. Generating a new token may revoke a token you're using elsewhere (IDE, CI/CD, other tools).

If You Don't Have a GoCode Token Yet

Step 1: Navigate to the GoCode Portal

Go to: <https://caas.open-webui.godaddy.com>

Sign in with your GoDaddy SSO credentials.

Step 2: Open API Keys

Click on **Workspace** then **API Keys**.

Step 3: Create a New API Key

1. Click the **+** button to create a new API key
2. Enter a **Key Alias** (a name for your reference)
 - The alias must be unique across all GoDaddy users
 - If you get an error, choose a different alias name
3. Optionally configure:

Setting	Description
Maximum Budget (USD)	Limit spend per key
Reset Budget Duration	How often the budget resets (e.g., 24h, 30d, 3600s)
Expiration Duration	When the key auto-expires (default: 30d, cannot be overridden)

4. Click **Create**

Step 4: Copy Your Key Immediately

After creation, you'll see the full secret key (e.g., `sk-fn6NKp...`).

Warning: This is the **ONLY** time the full key will be visible. Copy it immediately and store it securely.

Token Format

```
sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

All valid GoCode tokens begin with the `sk-` prefix.

Key Expiration

By default, keys expire after **30 days**. Unless you specify a different duration during creation, you'll need to generate a new key once it expires.

Managing Your Keys

In the GoCode portal under **My API Keys**, you can:

- View a shortened version of your secret key
- See status (ACTIVE), expiry, and budget info
- Track usage
- Delete unused or compromised keys

Using the Token with CLI Tools

For Moon Units, your token is passed during `mu init`. For direct CLI usage:

```
# Claude Code
export GOCODE_API_TOKEN=sk-your-token-here
export ANTHROPIC_MODEL=claude-sonnet-4-5-20250929
ANTHROPIC_BASE_URL=https://caas-gocode-prod.caas-prod.prod.onkatana.net \
  ANTHROPIC_AUTH_TOKEN=$GOCODE_API_TOKEN \
  claude

# Aider
ANTHROPIC_BASE_URL=https://caas-gocode-prod.caas-prod.prod.onkatana.net \
  ANTHROPIC_API_KEY=$GOCODE_API_TOKEN \
  aider --model claude-3-7-sonnet-20250219

# Codex (OpenAI)
OPENAI_BASE_URL=https://caas-gocode-prod.caas-prod.prod.onkatana.net/v1 \
  OPENAI_API_KEY=$GOCODE_API_TOKEN \
  codex -m gpt-5-codex
```

Model Access

Your token and org policy control which models you can call. If a manifest's `model` field specifies a model your token isn't authorized for, stages may hang or time out. You can switch models dynamically by exporting a new value for `ANTHROPIC_MODEL`.

Where the Token is Stored

After running `mu init`, the token is written to `~/.config/mu/mu.env` as part of the `MOONUNIT_*` environment variables.

Best Practices

- Always store your secret keys securely
- Set budget caps to prevent overuse or abuse
- Delete unused or compromised keys immediately

- Remember: keys expire after 30 days by default — regenerate when needed
- Never share your keys with others

Troubleshooting

Problem	Solution
401 Unauthorized	Verify token is valid and you're on VPN
Claude Code 400 errors about <code>input_examples</code> or <code>cache_control</code>	<code>export CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS=1</code>
Key alias already taken	Choose a different, unique alias — aliases are global across all users
Key expired	Generate a new key (default expiry is 30 days)
Claude Code persistent errors	Try in order: (1) <code>export CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS=1</code> (2) <code>rm -rf ~/.claude/statsig</code> (3) Add <code>127.0.0.1 statsig.anthropic.com</code> to <code>/etc/hosts</code> (4) <code>rm -rf ~/.claude ~/.claude.json</code> (5) <code>export DISABLE_PROMPT_CACHING=1</code>

Support

For GoCode support or help integrating additional CLIs, contact: **#gocode-alpha** on Slack.

Reference

- [GoCode Generated Keys + CLI Tools](#)
- [GoCaaS — Generate Your Developer API Keys](#)

Updating Credentials Later

Run `mu init` again. Press Enter on any field to keep its existing value.

Manual Configuration (Advanced)

Edit the env file directly:

```
vi ~/.config/mu/mu.env
```

Step 7: Set Your AWS Profile

The Moon Unit Docker image is hosted in ECR and requires a non-PCI AWS account:

```
export AWS_PROFILE=your-dev-profile
```

Warning: Credentials for organizations outside non-PCI will result in a **403 Forbidden** error when pulling the Docker image.

Step 8: Run Your First Mission

Option A: Watch Jira (Continuous Polling)

This polls Jira for tickets with a specific label and automatically launches missions:

```
mu watch --plan bug-fix \  
  --jira-project MYPROJECT \  
  --jira-label my-mu-label \  
  --repo-rw https://github.com/my-org/my-repo
```

What happens:

1. First run pulls the Docker image from ECR (~1 minute)
2. **mu** polls Jira every 30 seconds for issues with your label
3. When a matching issue is found, a mission launches automatically
4. Press **Ctrl+C** to stop watching

To re-run a processed ticket, remove the **<label>--processed** label in Jira.

Option B: Launch a Single Mission

Run once against a specific Jira issue:

```
mu launch --plan bug-fix \  
  --repo-rw https://github.com/my-org/my-repo.git \  
  --input-jira MYPROJECT-123
```

The TUI runs autonomously through completion.

Option C: Launch from a YAML Manifest

```
mu launch my-manifest.yml --mount-workspace
```

Use **--mount-workspace** to see output files on your host machine.

Verifying Everything Works

Check	Command
<code>mu</code> is installed	<code>mu version</code>
Credentials are configured	<code>ls ~/.config/mu/mu.env</code>
Docker is running	<code>docker info</code>
AWS credentials work	<code>aws sts get-caller-identity</code>
GitHub CLI works	<code>gh auth status</code>

Updating Moon Units

Automatic Update

`mu` checks for nightly updates before each `mu launch`. You'll be prompted if an update is available.

Manual Update

```
mu update
```

This pulls the latest Docker image and checks for a newer binary.

Disable Auto-Update

```
export MU_SKIP_UPDATE=1
# or
mu launch --no-update-check my-manifest.yml
```

Troubleshooting

Problem	Solution
<code>403 Forbidden</code> on Docker pull	Ensure <code>AWS_PROFILE</code> points to a non-PCI account
"'mu' could not be verified" on macOS	Run <code>xattr -d com.apple.quarantine mu</code>
<code>mu init</code> hangs	Ensure you have network access to GoDaddy internal services
Mission sits in "pending" for a long time	Docker image is being pulled (~1 min first time) or repo is being cloned
GoCode token invalid	Verify with <code>echo \$MOONUNIT_GOCODE_TOKEN</code> — must start with <code>sk-</code>

Problem	Solution
Jira issues not being picked up	Check label spelling matches exactly, and Jira credentials are valid

Uninstalling

```
# Remove the binary
rm ~/.local/bin/mu

# Remove configuration
rm -rf ~/.config/mu/
```